



Mansoura University
Faculty of Computers and Information
Department of Computer Science



Spring 2026

[SWE143] Software Project Management

Lecture 07: **Estimation: Function Points**

Level 300: software engineering program

Muhammad Haggag Zayyan, Ph.D

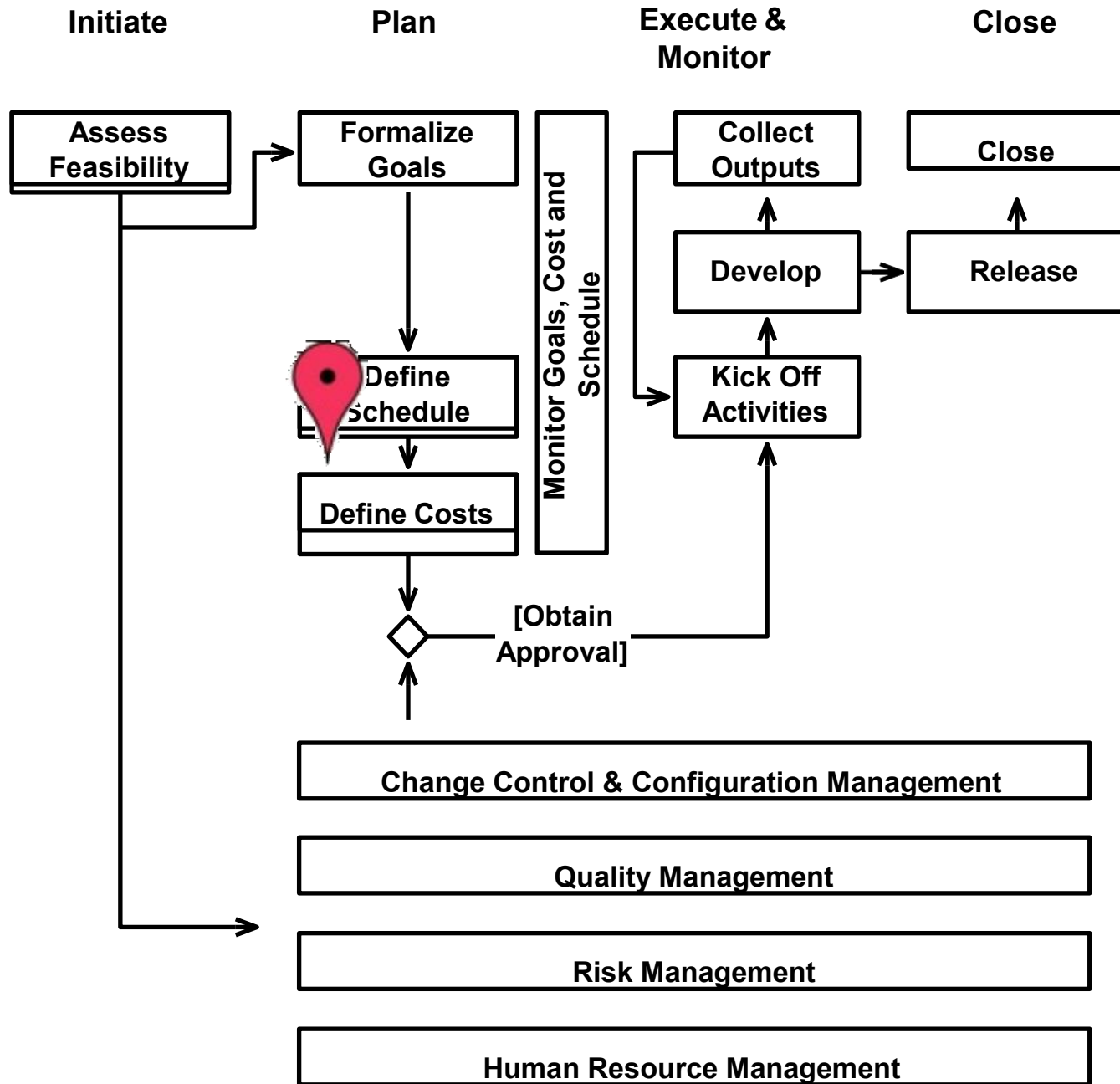
(Revisit) Approaches to Estimation

- **Expert Judgement** is “quick and dirty” and based on experience. It can be applied either top-down or bottom-up
- **PERT (Program Evaluation and Review Technique)** takes into account the probabilistic nature of estimations
- **Algorithmic Techniques** provide estimations by measuring specific qualities of a system and applying algorithms (Function Points, COCOMO, WebObjects)

Algorithmic Techniques

Goals of the Unit

- Understand the basic principles behind algorithmic techniques and, more in details, about Function Point Estimation



Introduction

- Goal: find a way to systematically determine the effort/duration required for an (arbitrary) task/project
- Ideally:
 - Identify a set of **measurable** characteristics of a project that determine the project's **effort/duration**
 - Define a function that, given the characteristics mentioned above, computes the **effort/duration**

$$f(x_1, \dots, x_n) = e$$

Problem: how do you find $f, x_1, \dots, x_n$?

Solution

- Look at existing projects/datasets; each project is represented by a vector:

$$\langle a_1, \dots, a_n, \textit{effort} \rangle$$

- Find correlations between (some of the) variables in the datasets:

$$f(a_1, \dots, a_k) \propto \textit{effort}$$

- Find appropriate measurement means for the variables at the beginning of a project (so that we can apply the function to a new project)

Variables can be:

Size-related:

- Lines of code (for software)
- Number of components/modules
- Number of features or requirements

Complexity-related:

- Algorithmic complexity
- Interdependencies
- Technical novelty

Resource-related:

- Team size
- Experience levels
- Availability of tools or infrastructure

Discussion

- Advantages:
 - **Replicable:** Apply the same model to similar projects.
 - **Objective:** reduces bias by relying on data-driven estimation (not opinion-based, such as expert judgment “**Subjective**”).
- Limitations of the models:
 - **Data dependency:** Small or noisy datasets lead to unreliable models
 - **Limited accuracy:** Model quality depends on the quality of training data.
- Limitations of their application:
 - **High resource cost:** Collecting data requires time, effort, and expertise
 - **Poor portability:** Models may not transfer well across different domains (e.g., software vs. construction)
 - **Input data quality:** Incorrect input data leads to incorrect estimates, regardless of model quality

Main Techniques

- Function Points (FP)
 - **Function-based size model** : Estimates effort based on **what** the system does, **not how** it's built.
 - It measure the number and complexity of the functions to develop
 - **FP is converted using productivity rates:**
 - **Effort** = FP × productivity rate (e.g., hours per FP)
 - **Duration** = Effort ÷ team capacity
 - It requires a critical analysis of the requirements to identify functions, inputs/outputs, and their complexity for Function Point estimation.
- Constructive Cost Modeling (COCOMO)
 - **LOC-based size model**: It estimates effort, duration, and team size based on the (**presumed**) size of a system in source lines of code (**SLOC**)
 - Different families of models – different versions
- Sometime used in conjunction (FP to get the system size; COCOMO to do the rest)
 - FP → SLOC conversion: Estimate size using FP, then input to COCOMO.
 - Combines strengths: Functional abstraction + proven cost modeling.

Function Based Metrics [FP]

- Advantages:
 - Functional complexity can be estimated when requirements are ready
 - It does not depend upon a specific programming language
- Disadvantages and critiques:
 - **More difficult to link to productivity:** FP does not clearly show how fast a team works.
 - **Does not directly map to time or effort:** It cannot directly tell project cost or completion time.
 - **More difficult to measure than size metrics:** Counting FP is harder than counting lines of code. (*requires analyzing inputs, outputs, files, interfaces, and processing logic*)

Size Metrics [SLOC]

- Advantages:
 - They can be precisely measured (once you have the source code and adopt coding standards)
 - Measuring tools available
 - More easily linkable to productivity
- Disadvantages and critiques:
 - They are known precisely when the system is built (a bit **too late** if you want to use them for estimation)
 - Sensitive to programming language, coding standards
 - 1 FP in Python may take 20 lines, but in C++ it might take 100 — making SLOC less portable across technologies.
 - **They do not measure the complexity of code**
 - A system with fewer lines can still be more complex, but SLOC won't reflect that

What is a line of code?

Lines of Code (LOC) can be misleading as a metric. The same functionality can be written in different styles—multi-line if-else, ternary, or one-line code.

Although all do the same task, the number of lines changes, so LOC does not accurately measure productivity or complexity

```
if (a == 1) {  
    return true;  
}  
else {  
    return false;  
}
```

```
return a == 1 ? true : false
```



```
if (a == 1) { return true; } else { return false; }
```

Source lines of code (SLOC)

Source lines of code (SLOC), also known as lines of code (LOC), is a software metric used to measure the size of a computer program by counting the number of lines in the text of the program's source code

- **Physical** SLOC is a count of lines in the text of the program's source code **excluding comment lines**.
- **Logical** SLOC attempts to measure the number of executable "statements"

```
for (i = 0; i < 100; i++) printf("hello"); /* How many lines of code is this? */
```

In this snippet of C code we have:

- 1 physical line of code (LOC),
- 2 logical lines of code (LLOC) (for statement and printf statement),
- 1 comment line.

Source lines of code (SLOC) – Limitation

how LOC is measured differently

Depending on the programmer and coding standards, the above "line" of code could be written on many separate lines:

```
/* Now how many lines of code is this? */  
for (i = 0; i < 100; i++)  
{  
    printf("hello");  
}
```

In this snippet of C code we have:

- 4 physical lines of code (LOC): is placing braces work to be estimated?
 - Braces and formatting: Are { and } counted as work? Some coding styles put them on new lines; others don't.
- 2 logical lines of code (LLOC): what about all the work writing non-statement lines?
 - LLOC only counts executable statements and does not include the effort involved in formatting or code structure. Braces, spacing, and layout are ignored.
- 1 comment line: tools must account for all code and comments regardless of comment placement.
 - comments contribute to code maintainability and developer effort.

Source lines of code (SLOC) – Limitation

how LOC is measured differently

- **Code density** refers to how “packed” or “spread out” code is in terms of physical lines versus logical statements.
 - A single logical statement can be written across multiple physical lines for readability, or multiple logical statements can be compressed into one physical line
 - Code density shows LOC is **not a reliable** measure of “amount of work”.
- **Tool inconsistency** means different software tools may count LOC differently because there is no single universal standard,
 - **including or excluding** blank lines, including or excluding comments, and treating braces {} as lines or ignoring them.

Productivity Examples

- **From Sommerville, Software Engineering:**

- **Real-time embedded systems:**

- * 40-160 LOC/P-month

Person-month

- * a developer can typically produce between 40 and 160 lines of code (LOC) in one month.

- **Systems programs:**

- * 150-400 LOC/P-month

- **Commercial applications:**

- * 200-900 LOC/P-month

- **An example:**

- Red Hat 7.1 is about 30 MSLOC, it would have required **8,000** person years if developed according to conventional proprietary practices **~313 LOC/P-month**
 - Debian 3.1 consider one of the largest software collections in the history of humanity, if not the largest one. In order to create their 196 MSLOC, **50,000** person-years would be necessary. **~327 LOC/P-month**
[release Date: June 2005 - source package: 8,141]

Limitations of LOC as a Productivity Measure

Why LOC/person-month can be misleading

- LOC does not reflect real productivity or value
- Few lines of code can perform complex tasks, while many lines may do simple work
- Modern development uses frameworks/libraries, reducing the need for writing code
- Much effort goes into design, testing, debugging, and integration—not just coding
- Different programming languages produce different LOC for the same task (e.g., Python vs C)

Therefore, LOC is not a reliable measure of productivity.

So, fewer LOC does not mean low productivity, and more LOC does not mean high productivity.

Function Points

Function Points

- Based on a combination of program characteristics derivable from the requirements (function-based metrics)
 - FP estimations are based on 19 different characteristics of a system, five of which refer to the functional characteristics of a system and 14 of which refer to nonfunctional aspects.
- First proposed by Albrecht in the 1970s
- First application in business systems (e.g., financial software)
- IFPUG trains users on applying the FP technique and promotes standards.
 - [The International Function Point Users Group](#) (IFPUG) is a US-based organization with worldwide chapters of Function point analysis metric software users. It is a non-profit, member-governed organization founded in 1986 –support its diffusion, application, and evolution (International function point user group [Wikipedia])

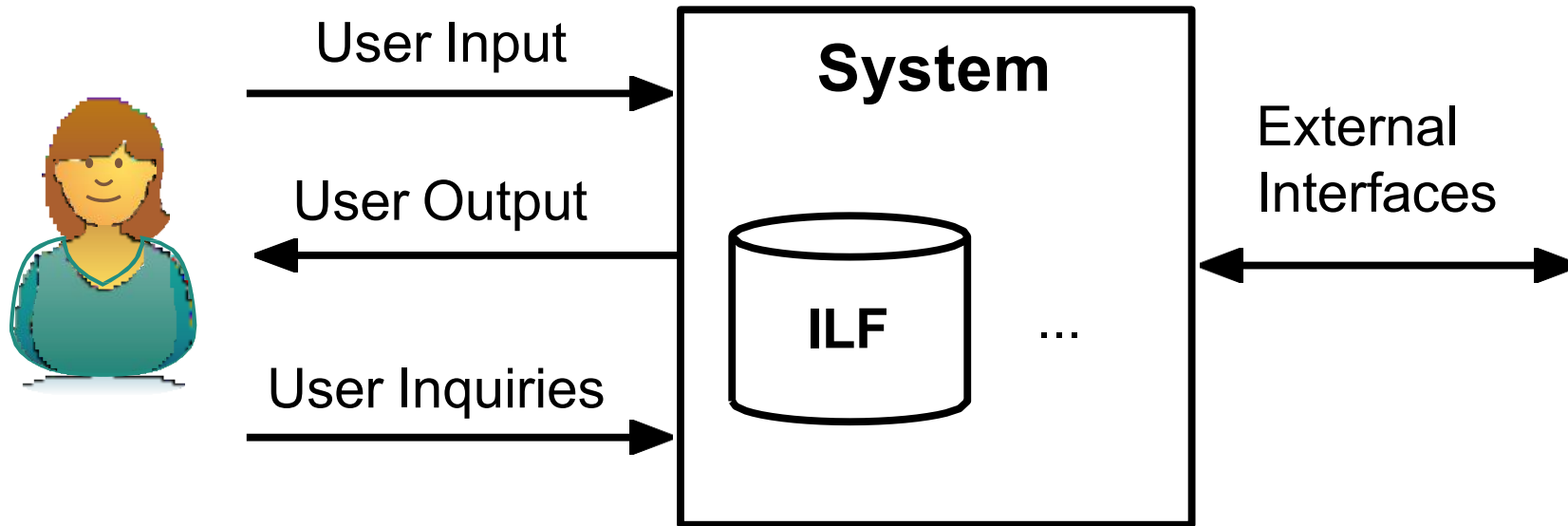
The Process at a Glance

1. Define the **boundary** of your analysis
2. Measure a set of characteristics related to the system's **functional requirements** and use a formula to get the **Unadjusted Function Points** (UFP)
3. Measure a set of characteristics related to the **non-functional requirements**: compute the **value adjustment factor** (VAF)
4. Compute the **(Adjusted) Function Points** (FP):
$$FP = f(VAF, UFP)$$

UFP: Inputs (5)

- **User inputs:** the total number of user inputs and their complexity (e.g., forms) - Modifies internal logical files.
- **User outputs:** the total number of user outputs and their complexity (e.g., output screen) - Usually reflects processed or calculated data.
- **User inquiries:** A paired input and output used only to retrieve information — it **does not change any internal data**. (e.g., Typing a word into a search bar and seeing a list of matching results).
- **Internal Logical Files:** the total number of files created and used dynamically by the system
- **External interfaces:** the total number of external files that connect with the software to an external system. For instance, if the software communicates with a device, 1 external interface

Count number of each, classified as Easy, Average, or Complex



VAF: Inputs (14)

Value Adjustment Factor (VAF) = measure of non-functional requirements

1. The system requires reliable saves and backups?
2. The system requires communication of data?
3. Elaboration is distributed?
4. Performance is a critical factor?
5. The system will work on a well known environment?
6. The system requires live data entry?
7. Data entry is articulated in complex transactions?
8. ILF are updated on-line?
9. Data are complex?
10. Internal elaboration is complex?
11. Code must be reusable?
12. Project includes also installation and data conversion?
13. System must be used in different organizations?
14. System must be easy to maintain and simple to use?

Answers from 0 (irrelevant) to 5 (extremely relevant)

Function Points Formulas

$$UFP = \sum_{i=1}^5 \left[\begin{matrix} k_i^E & k_i^A & k_i^C \end{matrix} \right] \cdot \left[\begin{matrix} n_i^E \\ n_i^A \\ n_i^C \end{matrix} \right]$$

where the index i runs over the **five** characteristics (user inputs, user outputs, . . .);

$$\left[\begin{matrix} n_i^S \\ n_i^A \\ n_i^C \end{matrix} \right]$$

is the number of easy (E), average (A), and complex (C) elements of type i

$$\left[\begin{matrix} k_i^S & k_i^A & k_i^C \end{matrix} \right]$$

is the weight assigned by the method to easy (E), average (A), and complex (C) elements of type i

- EI (External Inputs): 3/4/6, EO (External Outputs): 4/5/7, EQ (External Inquiries): 3/4/6, ILF (Internal Logical Files): 7/10/15, EIF (External Interface Files): 5/7/10 (Simple/Average/Complex weights as per IFPUG).

$$VAF = \frac{65 + \sum_{i=1}^{14} C_i}{100}$$

$$FP = VAF * UFP$$

$$FP = UFP \cdot (0.65 + 0.01 \cdot \sum_{i=1}^{14} C_i)$$

You count the number of functions of each type and complexity, multiply by their weights, and sum them up to get the **Unadjusted Function Point total**.

Formulas Comment

- **UFP, unadjusted function points** is the weighted sum of external inputs, external outputs, ...
- The value of the constants (k^E , k^A , k^C) is provided by the method
- **FP, (adjusted function points)** takes into account the general system characteristics (the 14 questions)
- FP in in the range **[0.65 UFP, 1.35 UFP]**
 - The answers are summed together and added to the magic number 65, thus yielding a value in the range 65–135, where 135 is obtained when the answer to all the questions is 5, since $65 + 5 * 14 = 135$.
 - If the system has no complexity influence, the estimate is scaled down by 35% (i.e., $UFP \times 0.65$).
 - If the system has maximum complexity, the estimate is scaled up by 35% (i.e., $UFP \times 1.35$).
 - This gives a $\pm 35\%$ adjustment window around the base effort estimate derived from the Unadjusted Function Points (UFP).

UFP: Program Characteristics

There are 5 different types of **function points**, also known as **Elementary Processes** (EP's) – Inputs, outputs, queries, internal files and external files. The 5 types of EP's can be grouped into 2 types of functions:

- Inputs, Outputs & Queries all qualify as **Transactional** Functions (**process**)
- Internal Files & External Files are distinguished as **Data** Functions (**storage**)

A Transactional Function is broken down into DET's and FTR's, while a **Data Function** is broken down into DET's and RET's. **Acronyms:**

- DET – **Data Element Type** is a unique user recognizable, non-repetitive field. – **Example:** A "name" field in a form
- FTR – **File Type Referenced** is a file type referenced by a transaction. An FTR must also be either an Internal or External file. – Example: A transaction accessing a **customer database**.
- RET – **Record Element Type** is a user recognizable sub group of data elements within an Internal or External File. – Example: Customer file with **RET1** = Customer Details (Name, Address, Phone) and **RET2** = Account Info (AccountID, Balance, Status).

UFP: Program Characteristics

The tables classify functions into complexity levels

- The counting of UFPs is performed by looking at the requirements
- Analysts look for the following elements:
 - **DET**: data element types
 - **FTR**: file type referenced
 - **RET**: record element type
- Tables convert the numbers above into the five characteristics
- Training is required to apply the technique accurately

From the table specifically:
If RET = 1 and DET is between 1 and 19 → Low
A “Low” data function might be:

- A small file or table
- With only a few columns (fields)
- And simple structure (not many subgroups of records)

Data Functions

ILF/EIF		DET	
RET	1..19	20..50	> 50
0..1	Low	Low	Average
2..5	Low	Average	High
>5	Average	High	High

Transactional Functions

UI		DET	
FTR	1..4	5..15	> 15
0..1	Low	Low	Average
2	Low	Average	High
> 2	Average	High	High

UO/EQ		DET	
FTR	1..5	6..19	> 19
0..1	Low	Low	Average
2..3	Low	Average	High
>3	Average	High	High

FP: Program Characteristics

- Joel Henry (Introduction to SPM) proposes a “practical” approach to the counting:
 - UI: number of dialog boxes classified as simple, average, and complex
 - UO: number of reports, classified as simple, average, and complex
 - UI: total number of user inputs that generate a software response
 - ILF: number of files and complex internal data structures, classified as simple, average, and complex
 - EIF: number of external files and connected device classified as simple, average, and complex
- Rule of the thumb, it provides a rough estimation

What do you do once you have FP?

- An FP estimation measures with a number the “size” of a system which has to be built
- You can now:
 - Relate FP directly to effort, through productivity measures, if your organization keeps this kind of data:
 - * FP per man month (Individual productivity)
 - * FP per calendar month (team delivery speed)
 - Convert FP to SLOC and use another estimation method (e.g. COCOMO)
 - Use it to cross-check or validate estimations provided by other types of estimations
 - **Example:** After using a size-based method like COCOMO, you can cross-check by estimating the FP count for the system and applying FP-specific productivity metrics.

If: 5 developers

Each = **10 FP per man-month**

Then:

- Theoretical capacity = 50 FP per man-month
- But actual delivery (calendar month) might be: 35 FP/month (due to meetings, integration, delays)

Function Points Productivity Metrics

- Each company should start its own productivity measuring program
- Some “general” data and studies available (see, e.g., <http://www.softwaremetrics.com/Articles/estimatingdata.htm>)
- Productivity changes with system size (<http://www.softwaremetrics.com/Articles/estimatingdata.htm>):
 - 1.3 hours/FP for a system of 50 FP
 - 12.1 hours/FP for a system of about 7000 FP
 - 133.6 hours/FP for a system of about 15000 FPs

As the system **size** increases, the average **effort** per Function Point rises due to higher design complexity, more interdependencies, greater communication overhead, and additional quality assurance tasks.

Relationship FP/SLOC

- FP can also be related to SLOC through FP/SLOC metrics
 - FP measures functionality
 - SLOC measures code size
- This allows to:
 - This allows use of algorithmic models like **COCOMO** to estimate effort, cost, and time.
 - Measure progress using SLOCs

- **Low (simple systems)** → **lowest** SLOC/FP, **high** productivity, simple code
- **Medium (typical systems)** → average SLOC/FP, normal productivity
- **High (complex systems)** → highest SLOC/FP, low productivity, complex code

Source: <http://www.qsm.com/resources/function-point-languages-table>

Language	Avg	Median	Low	High
ABAP (SAP)	18	18	16	20
Access *	36	38	15	47
Ada	154	-	104	205
Advantage	38	38	38	38
Assembler *	209	203	91	320
C *	148	107	22	704
C++ *	59	53	20	178
C# *	58	59	51	66
Clipper *	40	39	26	53
COBOL *	80	78	8	400
ColdFusion	68	56	52	105
Cool:Gen/IEF *	37	35	10	180
Culprit	51	-	-	-
Datastage	67	79	16	85
DBase IV	52	-	-	-
Easytrieve+	33	34	25	41
Excel	47	46	31	63
FORTRAN	90	118	35	-
J2EE *	57	50	50	67
Java *	55	53	9	214

FP: Final Considerations

- **Advantages**

- **Objective measure** → same system gets same size
 - Example: Two teams estimate the same app → FP gives similar result
- **Helps pricing** → easier to agree cost
 - Example: “This system = 100 FP → cost decided per FP”
- **Early estimation** → from requirements only
 - Example: Before coding starts, we still estimate effort
- **Better understanding** → clarifies system scope
 - Example: Breaking system into inputs, outputs, files helps understand features

- **Disadvantages**

- **Needs training** → not easy for beginners
 - Example: Wrong DET/RET counting gives wrong FP
- **Rigid if used for pricing** → changes become costly
 - Example: Client adds new feature → price increases
- **Hard with requirement changes**
 - Example: After agreement, new module added → re-estimation needed again

T&M (Time & Material) is a **non-rigid** pricing model where the client pays for actual time spent and work effort/resources (like coding, testing, design) used, and changes in requirements are handled smoothly by adjusting effort and cost dynamically without strict contract rework.

Thank You

